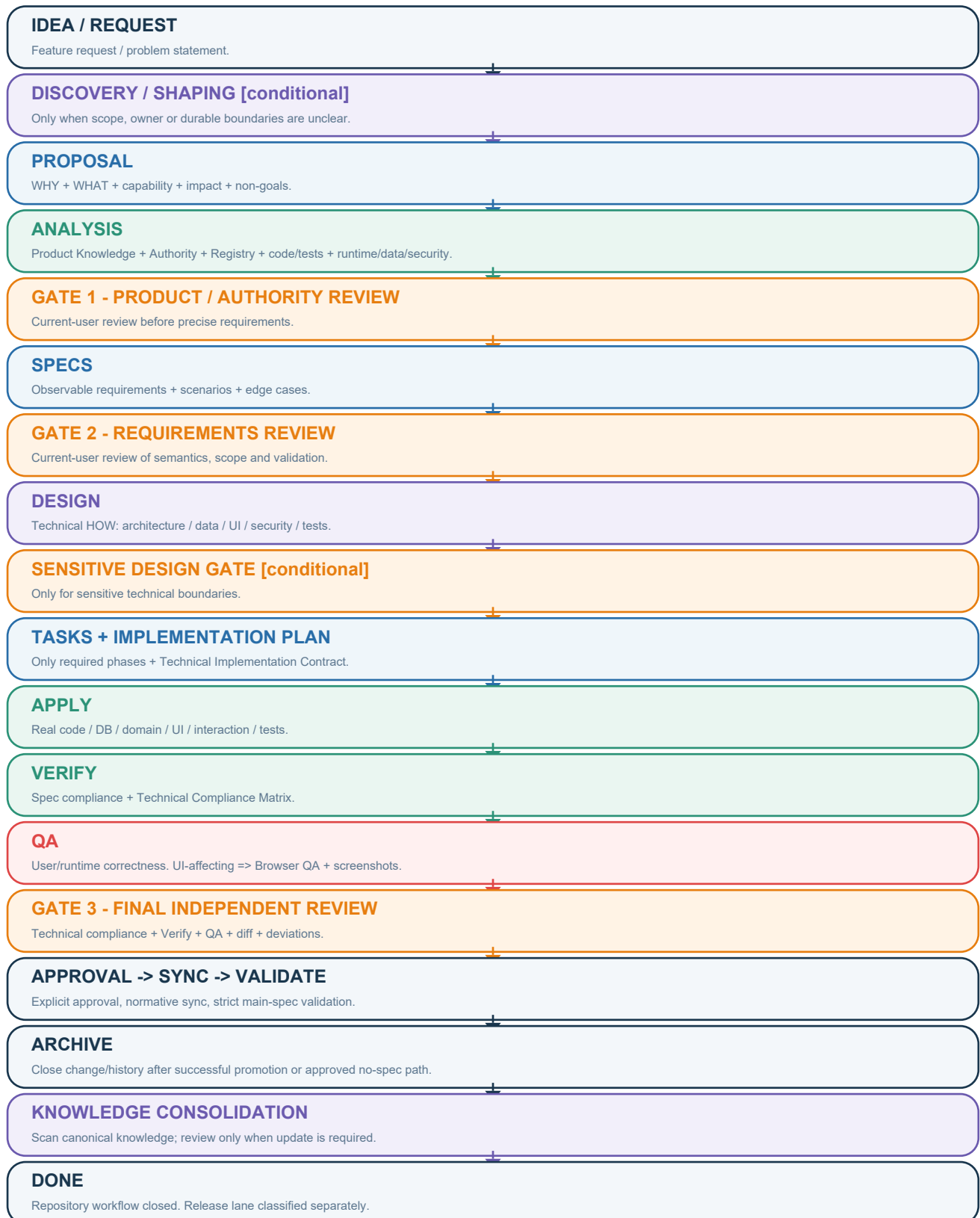


YuTa Workflow v3 - End-to-end

Approved repository workflow. Release/deploy remains a separate operational lane.

Page 1/8



Stage reference - Product to Design

Purpose, key work, outputs and stop conditions.

Page 2/8

STAGE	PURPOSE	KEY WORK	OUTPUT / STOP
IDEA	Capture a new need.	Problem, opportunity, desired outcome.	Input only. No authority created.
DISCOVERY*	De-risk before OpenSpec.	Read-only shaping for unclear owner, cross-module scope, provider or unfamiliar runtime/data boundary.	Question, bounded request, missing decision or no change.
PROPOSAL	Bound WHY + WHAT.	Why now; capability paths; impact; non-goals. No technical design.	proposal.md
ANALYSIS	Check whether YuTa is understood correctly.	Product Knowledge, Authority, Module Registry, ADRs, code/tests, runtime/data/security, conflicts and unknowns.	analysis.md. Conclusion: READY_FOR_SPECS / BLOCKED_NEEDS_REVIEW / NO_SPEC_BEHAVIOR_CHANGE.
GATE 1	Product/authority review.	Exact proposal + analysis, hashes, conflicts and questions requiring decision.	01-analysis-review.md. Stop until explicit current-user approval.
SPECS	Define observable behavior.	Requirements, scenarios, SHALL/MUST/WHEN/THEN and edge cases. No implementation design.	specs/<capability>/spec.md
GATE 2	Review requirement semantics.	Exact spec hashes, strict OpenSpec validation and ambiguity check.	02-specs-review.md. Stop until explicit current-user approval.
DESIGN	Choose technical HOW.	Architecture, data flow, permission enforcement, UI strategy, migration/rollback, tests and trade-offs.	design.md when required by the artifact graph.
DESIGN GATE*	Review sensitive decisions.	Authorization/security, data owner/runtime, destructive migration, payment/fiscal, legal/privacy, provider, POS integrity or durable cross-module boundary.	02b-design-review.md. Stop before Tasks/Apply.

* conditional: Discovery and the sensitive Design Gate appear only when applicable; the approved no-spec path omits Specs and Gate 2.

Stage reference - Implementation to DONE

Implementation, verification, QA, promotion and knowledge loop.

Page 3/8

STAGE	PURPOSE	KEY WORK	OUTPUT / STOP
TASKS	Create executable plan.	Select only needed phases. Add checkboxes and phase Technical Implementation Contracts.	tasks.md. Apply blocked if owner/boundary/authority unresolved.
APPLY	Implement approved scope.	Execute selected phases in dependency order. Complete tasks only with outcome + compliance evidence.	Product changes + completed tasks. Stop if scope/boundary must change.
VERIFY	Prove technical correctness.	Requirement mapping, tests, typecheck/build, OpenSpec validation, architecture/security/schema/migration, diff and Technical Compliance Matrix.	VERIFY PASS only when Specs/Design match and applicable contract rows PASS.
QA	Prove user/runtime correctness.	UI: real browser, responsive, roles/states, accessibility, visual and screenshots. Non-UI uses applicable QA or NOT_APPLICABLE.	QA report/evidence when applicable. PASS / FAIL / BLOCKED_BY_ENVIRONMENT / NOT_APPLICABLE.
GATE 3	Final independent review.	Technical Compliance, Verify, QA, scoped implementation diff, deviations, hashes and lifecycle truth.	03-final-review.md. Ready only with required evidence PASS.
APPROVAL	Governance decision.	Current user explicitly approves exact Gate 3 and authorizes sync/archive. Hashes remain intact.	Gate 3 APPROVED + sync authorization.
SYNC	Mechanical normative promotion.	Merge only selected approved delta specs into main specs; inspect exact diff.	Normative spec path. Sync creates no approval.
VALIDATE	Protect normative tree.	Strict main-spec validation and diff review; safe exact rollback on failed promotion.	Validated main spec or stop/rollback.
ARCHIVE	Close change history.	Archive only after valid promotion/no-spec path and complete tasks/evidence.	openspec/changes/archive/...
KNOWLEDGE	Reconcile current knowledge.	Scan materially affected Product Knowledge, routing, Registry/lifecycle, ADR, page pack, CURRENT_STATE and resolved NEEDS REVIEW.	NO_UPDATE_REQUIRED -> DONE, or Knowledge Review packet -> stop.
DONE	Close repository workflow.	All repository governance terminal. Classify RELEASE_FOLLOW_UP separately.	Workflow status DONE.

Automation, integrity and escalation

What Codex automates, where human review is mandatory, and how unclear scope is routed.

Page 4/8

\$yuta-run-change

Start/resume an active change and run only to the next human gate. Owns conditional Discovery, planning, Gates, phased Apply, Verify and QA. Never sync/archive.

\$yuta-finish-change

After explicit Gate 3 approval + sync/archive authorization: integrity recheck -> sync -> validate -> archive -> Knowledge Consolidation. Also resumes approved post-archive Knowledge Review.

Review packet integrity

Every gate records exact SHA-256 for reviewed paths/bytes. Resume/finish recompute recorded path sets and hashes. Any reviewed addition, removal, rename or byte change invalidates the packet and stops for re-review. Tests, PR status or prior assistant text never replace explicit approval.

GATE 1

Proposal + Analysis
Product / authority review
01-analysis-review.md

GATE 2

Delta Specs
Requirements / edge cases
02-specs-review.md

GATE 3

Implementation + Verify + QA
Final independent review
03-final-review.md

DISCOVERY / SHAPING escalation

When owner, cross-module scope, provider behavior or unfamiliar runtime/data boundaries are unclear, inspect current authority and implementation read-only. Ask for the missing decision, produce a bounded request, or stop without creating a change. Do not invent a mandatory artifact or routing protocol.

Adoption / resume rule

Preserve existing in-flight artifacts byte-for-byte unless an approved revision authorizes edits. Resume at the earliest missing, unapproved, invalidated or changes-requested gate. Later artifacts never bypass earlier human review.

Tasks + Technical Implementation Contract

Each selected phase resolves current scoped authority and records compliance evidence.

Page 5/8

Contract fields for every phase actually used

Affected boundary and canonical owner; root + nearest scoped AGENTS.md; applicable current authority paths; resolved constraints; intended files/packages; targeted checks; completion evidence. Reference sources - do not duplicate repository rules in workflow text.

Foundation / Data

Resolve current runtime/database owner, schema/migration consistency, tenant/isolation constraints and required migration/schema evidence.

Service / Domain

Resolve trusted scope, authorization order, boundary validation, domain purity and canonical contract/repository ownership.

UI / Components

Resolve current component/client ownership, application/scoped UI rules, design-system authority and browser boundary constraints.

Interaction / States

Resolve current mutation/form conventions, applicable response states, accessibility and canonical state ownership.

Integration / Regression

Resolve targeted/broader regression, build/typecheck, architecture/security/migration checks and scoped dirty-worktree attribution.

APPLY completion rule

Apply must not start while a required technical owner, boundary, authority or constraint is unresolved. Execute selected phases in dependency order. A task is complete only when its implementation outcome and phase compliance evidence both exist. If implementation requires a changed permission, contract, API, canonical owner, cross-runtime behavior or durable boundary, STOP and return to the applicable Design/Product/authority gate. Do not rewrite Specs/Design to fit code.

VERIFY and QA protocol

Technical correctness and user-facing correctness are separate evidence layers.

Page 6/8

VERIFY - technical

Requirement/scenario -> implementation
Targeted + broader tests
Typecheck / build
OpenSpec strict validation
Architecture/security
Schema/migration checks
Scoped diff / deviations
Technical Compliance Matrix

QA - user/runtime

UI: browser interaction
Responsive
Role/edit/read-only
Loading/error/success
Keyboard + accessibility
Visual consistency
Regression
Screenshots

TECHNICAL COMPLIANCE MATRIX

technical rule/constraint -> authoritative source -> affected implementation -> test/check/evidence -> PASS/FAIL. VERIFY may pass only when Specs/Design match implementation and every applicable Technical Implementation Contract item passes.

UI_AFFECTING rule

Visible UI, interaction, responsive layout, role/edit/read-only state, loading/error/success presentation or visual behavior => UI_AFFECTING=YES and BROWSER_QA_REQUIRED=YES. Gate 3 requires QA=PASS.

Non-UI rule

Backend/database correctness remains in VERIFY. Use an applicable non-browser QA plan or truthful NOT_APPLICABLE when no user/runtime QA dimension exists; do not require meaningless Browser QA.

Browser QA evidence

Use the real/local route and page-pack viewports; otherwise minimum desktop 1366x768 and mobile 390x844, adding tablet when meaningful. Store QA_REPORT.md, screenshot-manifest.md and actual hashed screenshots with viewport, role/state and scenario.

QA status and environment blocking

PASS | FAIL | BLOCKED_BY_ENVIRONMENT | NOT_APPLICABLE. UI changes cannot use NOT_APPLICABLE. A required environment block remains visible and cannot produce a ready Gate 3.

Finish and Knowledge Consolidation

Normative promotion, archive and post-archive review use separate guarded branches.

Page 7/8

NO-SPEC PATH - skip_specs: true

Gate 1 approval -> no Gate 2 -> no delta/main-spec promotion. Continue only applicable Design/Tasks/Apply/Verify/QA -> Gate 3 -> final approval -> archive -> Knowledge Consolidation. Never invent a requirement to satisfy validation.

BRANCH A - Active finalization

Preconditions:

- active change exists
- Gate 3 = AWAITING_HUMAN_REVIEW
- explicit final approval + sync/archive authorization
- recheck all reviewed path sets/hashes, including planning, earlier packets, implementation diff, Verify, Technical Compliance and applicable QA

Flow:

APPROVE -> SYNC -> VALIDATE -> ARCHIVE -> KNOWLEDGE SCAN

BRANCH B - Archived Knowledge Review

Preconditions:

- Gate 3 = APPROVED
- Finish outcome = COMPLETED
- Workflow = AWAITING_KNOWLEDGE_REVIEW
- no active change; recorded archive exists
- 04 review = AWAITING_HUMAN_REVIEW

Flow:

current-user Knowledge Review approval -> target path set + target hashes + proposed-diff hash -> apply exact approved documentation diff -> format/docs:check/architecture:check -> evidence -> DONE
Never rerun active integrity, sync or archive.

KNOWLEDGE SCAN

Inspect only materially affected current sources: Page/Module Product Knowledge, routing, Module Registry/lifecycle, ADR, page pack, CURRENT_STATE, resolved NEEDS REVIEW and newly discovered limitations/future work.

NO_UPDATE_REQUIRED

Record bounded reason + exact sources inspected. Workflow status DONE.

UPDATE_REQUIRED

Create 04-knowledge-consolidation-review.md with exact proposed diff, target path set and hashes. Do not edit canonical knowledge before approval.

Knowledge safety

Knowledge Consolidation cannot automatically approve Product Intent, change durable boundaries, ownership/permissions, promote lifecycle/readiness, rewrite normative specs or resolve NEEDS REVIEW by assumption. Knowledge Review approval is separate from Gate 3 approval.

Release lane + quick operational memory

Repository workflow finishes before deployment; deployment has separate evidence and authorization.

Page 8/8

REPOSITORY WORKFLOW DONE

Normative specs synced/validated when applicable, change archived and Knowledge Consolidation terminal. This is repository completion - not deployment proof.

RELEASE FOLLOW UP

Classify NOT_REQUIRED | REQUIRED | UNKNOWN. If REQUIRED, report runtime/environment, deployment/readiness evidence and post-deploy verification needs.

RELEASE / DEPLOY

Authorized operational action. Never auto-deploy from \$yuta-finish-change.

POST-DEPLOY VERIFY

Smoke / health / runtime checks on the real environment.

MONITOR / ROLLBACK

Observability, incident/recovery evidence and rollback when applicable.

Cách nhớ ngắn

Proposal = tôi muốn gì? | Analysis = tôi hiểu YuTa đúng chưa? | Specs = YuTa phải làm gì? | Design = code bằng cách nào?

Tasks = chia việc + technical contract | Apply = code thật | Verify = technical proof | QA = user/runtime proof | Gate 3 = final independent review

Sync = normative promotion | Archive = close history | Knowledge Consolidation = update current knowledge if needed